Mobil Uygulama Geliştirme ve React Native Ders Notları

BÖLÜM 1: MOBİL UYGULAMA DÜNYASINA GİRİŞ VE PAZAR DİNAMİKLERİ

1.1. Mobil Teknolojilerin Gelişimi ve Güncel Durum

Mobil cihazlar; teknolojik yakınsama, yüksek işlem gücü ve geniş ekran standartlarıyla birlikte günlük yaşamın ve iş süreçlerinin merkezine yerleşmiştir. Taşınabilir cihazların maliyetlerinin düşmesi ve kablosuz erişim teknolojilerinin (5G/WiFi 6) yaygınlaşması, mobil uygulama kullanımını masaüstü ve mobil web ortamlarının önüne geçirmiştir.

- **Kullanım İstatistikleri ve Eğilimler:** Güncel küresel verilerde mobil cihazların (akıllı telefonlar ve tabletler) toplam internet trafiğindeki payı %58–%62 bandında seyrederken, masaüstü cihazların payı %38 civarındadır. Türkiye pazarında mobil cihazların pazar payı %70'in üzerine çıkmıştır.
- **Uygulama Kullanım Süreleri:** Kullanıcılar mobil cihazlarda geçirdikleri vaktin %85–%90'ından fazlasını mobil uygulamalar içerisinde, kalan kısmını ise mobil web tarayıcılarında harcamaktadır.
- **Kurumsal ve Ticari Etki:** İşletmeler için mobil uygulamalar; marka sadakati, müşteri deneyimi (UX), doğrudan pazarlama kanalı (anlık bildirimler/push notifications) ve süreç otomasyonu için temel gereksinim haline gelmiştir.

1.2. Mobil Uygulama Kullanım Alanları ve Kategorileri

Mobil uygulamalar sundukları işlevselliğe göre çeşitli kategorilere ayrılır:

1. **Eğitim (EdTech):** Mikro öğrenme, esnek erişim ve oyunlaştırma (gamification) odaklı sistemler.
2. **Mobil Ticaret (M-Commerce):** Hızlı ödeme entegrasyonları (Apple Pay, Google Pay), kişiselleştirilmiş ürün önerileri ve AR (Artırılmış Gerçeklik) ile ürün deneme özellikleri.
3. **Sosyal Ağlar ve İletişim:** Gerçek zamanlı veri akışı, medya paylaşımı ve uçtan uca şifreli mesajlaşma altyapıları.
4. **Oyun Uygulamaları:** Uygulama mağazası gelirlerinin en büyük kısmını oluşturan, yüksek performanslı ve grafik odaklı yazılımlar.

5. **Eğlence ve Medya Akışı (Streaming):** İsteğe bağlı video/müzik (VOD), canlı yayın altyapıları.
6. **Yaşam Tarzı ve Sağlık (mHealth):** Giyilebilir cihazlar (Apple Watch, Wear OS) ve biyometrik sensör entegrasyonlu takip sistemleri.
7. **Finans ve Bankacılık (FinTech):** Yüksek güvenlikli (Biometric Auth, SDK), hızlı işlem odaklı finansal araçlar.
8. **Üretkenlik ve İş:** Belge yönetimi, proje takibi ve ekip içi iş birliği sistemleri.

1.3. Mobil Uygulama Mimari Türleri

A. Yerel (Native) Uygulamalar

- **Tanım:** Doğrudan hedef işletim sisteminin native dilleri ve SDK'ları kullanılarak geliştirilen uygulamalardır (iOS için Swift/Objective-C; Android için Kotlin/Java).
- **Avantajları:** Maksimum donanım ve GPU performansı, platform güncellemelerine anında erişim, yüksek güvenlik.
- **Dezavantajları:** İki ayrı kod tabanı (codebase) yönetimi, yüksek geliştirme ve bakım maliyeti, pazara sunma süresinin (Time-to-Market) uzun olması.

B. Web Tabanlı Uygulamalar

- **Tanım:** Mobil cihazların web tarayıcıları üzerinden çalışan, HTML5, CSS3 ve JavaScript ile geliştirilen duyarlı (responsive) web siteleridir.
- **Avantajları:** Kurulum gerektirmez, mağaza onay süreçleri yoktur, tek kod tabanıdır.
- **Dezavantajları:** Cihaz donanımına (GPS, Kamera, Bluetooth) erişim yok denecek kadar azdır, çevrimdışı çalışamaz, performans düşüktür.

C. İleri Web Uygulamaları (Progressive Web Apps - PWA)

- **Tanım:** Standart web teknolojileriyle yazılan fakat *Service Worker* ve *Web App Manifest* altyapısı sayesinde native uygulama benzeri deneyim sunan yapılardır.
- **Avantajları:** Çevrimdışı çalışma desteği, ana ekrana eklenebilme, düşük veri kullanımı, arka plan senkronizasyonu.
- **Dezavantajları:** iOS/Safari tarafında API kısıtlamaları (Bluetooth, arka plan konum takibi vb.), App Store dağıtım zorlukları.

D. Hibrit (Hybrid) Uygulamalar

- **Tanım:** Web teknolojileri (HTML, CSS, JS) ile geliştirilip yerel bir kılıf (`WebView`) içerisinde çalıştırılan uygulamalardır (Örn: Apache Cordova, Ionic).
- **Dezavantajı:** Arayüz çizimleri WebView üzerinde gerçekleştiği için performans yerel uygulamaların oldukça gerisindedir.

E. Çapraz Platform (Cross-Platform) Uygulamalar

- **Tanım:** Tek bir kod tabanı kullanılarak hem iOS hem de Android işletim sistemlerine çıktı üreten modern framework'lerdir.
- **Farkı:** WebView yerine, kodları doğrudan yerel derleme (native compilation) veya yüksek hızlı yerel köprüler/motorlar aracılığıyla doğrudan native UI bileşenlerine dönüştürürler. (Örn: React Native, Flutter).

BÖLÜM 2: DİL VE PLATFORM EKOSİSTEMİ

2.1. Mobil Uygulama Geliştirme Dilleri

- **Kotlin:** Google tarafından Android geliştirme için birincil (first-class) dil kabul edilen, null-safety özelliği yüksek modern statik dil.
- **Swift:** Apple ekosistemi (iOS, macOS, watchOS) için geliştirilmiş, hızlı, güvenli ve okuması kolay nesne yönelimli dil.
- **JavaScript / TypeScript:** Web dünyasının temeli olan, React Native gibi framework'ler sayesinde mobil geliştirmede yaygınlaşan dinamik/tip güvenli diller.
- **Dart:** Google tarafından Flutter framework'ü için özel geliştirilen, AOT (Ahead-Of-Time) ve JIT (Just-In-Time) derleme sunan dil.
- **C#:** Microsoft tabanlı, .NET MAUI (eski adıyla Xamarin) ve Unity oyun motoru ile mobil uygulama/oyun geliştirmede kullanılan dil.

2.2. Geliştirme Platformları ve Framework'ler

Platform / Framework	Geliştirici	Temel Dil	Mimari / Yaklaşım	Güncel Durum
React Native	Meta	JS / TypeScript	React Mantığı + Native UI Bileşenleri	Yaygın (New Architecture)
Flutter	Google	Dart	Özel Grafik Motoru (Impeller / Skia)	Yaygın
.NET MAUI (Xamarin)	Microsoft	C#	Native UI Bağlayıcıları	MAUI adıyla güncel
Apache Cordova	Apache	HTML/CSS/JS	WebView Kılıfı	Eski / Kullanım Oranı Düşük
Solar2D (Corona)	Açık Kaynak	Lua	2D Oyun Motoru	Özel Oyun Segmenti

---

2.3. Mobil Uygulama Mağazaları

- **Google Play Store:** Android uygulamalarının dağıtım merkezi. Geliştirici konsolu için tek seferlik ücret alır (Android App Bundle - `.aab` formatı zorunludur).
- **Apple App Store:** iOS/macOS uygulamaları dağıtım platformu. Yıllık üyelik modeli vardır; sıkı güvenlik ve tasarım inceleme (Review Guidelines) süreçlerine sahiptir.
- **Diğer Mağazalar:** Huawei AppGallery, Samsung Galaxy Store, Amazon Appstore.

BÖLÜM 3: REACT NATIVE İLE MODERN MOBİL UYGULAMA GELİŞTİRME

3.1. React Native Nedir?

React Native, Meta (Facebook) tarafından geliştirilen ve JavaScript/TypeScript kullanarak iOS ve Android platformlarına native (yerel) arayüz sunan açık kaynaklı bir framework'tür.

- **Temel Felsefe:** *"Learn once, write anywhere"* (Bir kez öğren, her yerde yaz).
- **Web React'ten Farkı:** React Native, HTML elemanları (`<div>`, `<span>`, `<p>`) yerine doğrudan native karşılığı olan bileşenleri (`<View>`, `<Text>`, `<Image>`) kullanır.

3.2. React Native Mimarisindeki Evrim: Eski vs. Yeni Mimari

Eski Mimari (The Old Architecture & The Bridge)

Geleneksel React Native mimarisinde JavaScript kodu ile Native (Java/Kotlin/Swift) tarafı bir **Bridge (Köprü)** üzerinden haberleşirdi.

```
None
[ JavaScript Thread ] <---> ( Asenkron JSON Bridge ) <---> [ Native / UI Thread ]
```

- **Sorun:** Köprü üzerindeki veriler JSON formatına dönüştürülüp asenkron taşındığı için yüksek hızlı kaydırmalarda (scroll), karmaşık animasyonlarda ve anlık veri aktarımlarında performans darboğazları (bottleneck) oluşuyordu.

Yeni Mimari (The New Architecture - Güncel Standart)

React Native, performansı native seviyesine çıkarmak için köprü yapısını tamamen kaldırmıştır. Güncel React Native mimarisi üç ana sütun üzerine kuruludur:

```
None
[ JavaScript Engine (Hermes) ]
        |
        ▼
[ JSI (JavaScript Interface) ] <---> Doğrudan Bellek & Nesne Erişimi <---> [
Native C++ / OS ]
        |
        ├─► Fabric (Yeni Render Motoru)
        └─► TurboModules (Tembel/Lazy Yüklenen Native Modüller)
```

1. **JSI (JavaScript Interface):** JavaScript kodunun C++ nesnelerine doğrudan referans vermesini sağlar. Köprüye ve JSON serileştirmeye gerek kalmadan native metodlar anında çağrılır.
2. **Fabric (Yeni Rendering Motoru):** Kullanıcı arayüzü çizimlerini senkronize edebilir. Ekran render performansını artırır ve UI gecikmelerini sıfıra indirir.
3. **TurboModules:** Native modüllerin uygulama açılışında hepsinin tek seferde yüklenmesi yerine, sadece ihtiyaç duyulduğunda (lazy loading) belleğe yüklenmesini sağlar. Bu da **App Startup Time** (uygulama açılış süresi) performansını ciddi oranda iyileştirir.
4. **Hermes JS Motoru:** Meta tarafından mobil için özel geliştirilmiş varsayılan JavaScript motorudur. Düşük bellek kullanımı, hızlı bayt kod derlemesi ve küçük APK/IPA boyutları sağlar.

3.3. Geliştirme Yaklaşımları: CLI vs. Expo (Modern Bakış)

Expo (Güncel ve Tavsiye Edilen Yaklaşım)

Eskiden Expo basit ve kısıtlı bir araç seti olarak görülürken, günümüzde **Expo Prebuild** ve **Custom Native Code** desteği sayesinde React Native geliştirmenin endüstri standardı haline gelmiştir.

- **Avantajları:**
  - Xcode veya Android Studio karmaşası olmadan hızlı kurulum.
  - **Expo Go** uygulaması ile fiziksel cihazda anında canlı test.
  - **EAS (Expo Application Services):** Bulut ortamında APK/IPA derleme ve mağazalara otomatik gönderme.
  - Native modüller yazabilme ve kütüphane desteği (Expo Config Plugins).

React Native CLI

Doğrudan native projeler (`ios` ve `android` klasörleri) üzerinde tam kontrol sağlayan geleneksel yöntemdir. Çok özel native SDK entegrasyonlarında veya mevcut native projelere React Native eklenirken tercih edilir.

3.4. Temel Bileşenler (Core Components)

React Native Bileşeni	HTML Karşılığı	iOS Native Karşılığı	Android Native Karşılığı
`<View>`	`<div>`	`UIView`	`android.view.ViewGroup`
`<Text>`	`<p>`, `<span>`	`UILabel`	`TextView`
`<Image>`	`<img>`	`UIImageView`	`ImageView`
`<ScrollView>`	`<div style="overflow:scroll">`	`UIScrollView`	`ScrollView`
`<TextInput>`	`<input type="text">`	`UITextField`	`EditText`
`<TouchableOpacity>`	`<button>`	`UIButton` (Custom)	`Button` (Custom)
`<FlatList>`	Liste Yapısı	`UITableView`	`RecyclerView`

3.5. Sayfa Düzeni (Layout) ve Styling

React Native'de stillendirme JavaScript nesneleri ve `StyleSheet.create()` API'si ile yapılır.

```javascript
import { StyleSheet, View, Text } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Merhaba React Native!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 20,
```

```
    fontWeight: 'bold',
  },
});
```

Flexbox Mantığı

- **Esas Yön (flexDirection):** Web'de varsayılan yön `row` iken, React Native'de varsayılan yön **`column`** (dikey) şeklindedir.
- **Önemli Hizalama Özellikleri:**
  - `justifyContent`: Ana eksendeki (main axis) hizalama (flex-start, center, flex-end, space-between).
  - `alignItems`: Çapraz eksendeki (cross axis) hizalama (flex-start, center, flex-end, stretch).
  - `flex: 1`: Bileşenin ekranda kaplayabileceği tüm esnek alanı kaplamasını sağlar.

3.6. Performanslı Liste Kullanımı: ScrollView vs. FlatList

ScrollView

Tüm alt bileşenlerini (children) uygulama yüklendiğinde hafızaya alır ve çizer. Az sayıdaki sabit içerikler için uygundur. Büyük listelerde bellek (RAM) tükettiği için uygulamayı kilitler.

FlatList (Önerilen)

Sadece ekranda o an görünen (ve yakınındaki) öğeleri render eder. Ekran dışına çıkan öğeleri bellekten boşaltır (**Virtualization**).

JavaScript
```
import { FlatList, Text, View } from 'react-native';

const DATA = [
  { id: '1', title: 'Birinci Öğeler' },
  { id: '2', title: 'İkinci Öğeler' },
];

export default function ListExample() {
  return (
    <FlatList
      data={DATA}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => (
        <View>
```

```
          <Text>{item.title}</Text>
        </View>
      )}
    />
  );
}
```

3.7. Navigasyon Yönetimi (React Navigation)

Mobil uygulamalarda sayfalar arası geçiş web'deki gibi URL tabanlı değil, bir **Stack (Yığın)** yapısı üzerindedir. Güncel standart **React Navigation** kütüphanesidir.

1. **Stack Navigator:** Sayfaları üst üste ekler (İleri / Geri geçişleri).
2. **Bottom Tabs Navigator:** Ekranın altında sabit sekme menüsü sunar.
3. **Drawer Navigator:** Yandan açılır çekmece menü yapısı.

BÖLÜM 4: MOBİL ARAYÜZLERDE KULLANILABİLİRLİK (UX) VE TASARIM

4.1. Mobil UX/UI ve Temel Prensipler

Mobil arayüz tasarımı, kısıtlı ekran alanı ve dokunmatik etkileşimler nedeniyle masaüstü tasarımlardan ayrılır.

- **Dokunmatik Alan Boyutları:** Dokunma hedefleri (butonlar, linkler) insan parmağının fizyolojisine uygun olarak minimum **44x44 pt** (veya 48x48 dp) boyutunda olmalıdır.
- **Başparmak Bölgesi (Thumb Zone):** Akıllı telefonların büyük çoğunluğu tek elle kullanılır. En kritik eylem butonları başparmağın rahat ulaşabildiği alt ve orta bölgelere yerleştirilmelidir.
- **Dikey (Portrait) Ekran Odaklılık:** Mobil etkileşimlerin %90'dan fazlası dikey tutuşta gerçekleşir.

4.2. Nielsen'in 10 Kullanılabilirlik Sezgisi (Heuristics)

1. **Sistem Durumunun Görünürlüğü:** Kullanıcıya ne olduğu hakkında anında dönüt verilmelidir (Örn: Yükleniyor ikonu / ActivityIndicator).

2. **Sistem ile Gerçek Dünyanın Eşleşmesi:** Kullanıcıya tanıdık gelen terimler ve ikonlar kullanılmalıdır.
3. **Kullanıcı Kontrolü ve Özgürlük:** Yanlış işlemleri geri alma (Undo) seçeneği sunulmalıdır.
4. **Tutarlılık ve Standartlar:** Uygulama içindeki renkler, buton stilleri ve terimler tutarlı olmalıdır.
5. **Hataları Önleme:** Kullanıcının hata yapmasını engelleyen form doğrulamaları tasarlanmalıdır.
6. **Hatırlamak Yerine Tanıma:** Bellek yükünü azaltmak için menüler ve seçenekler açıkça gösterilmelidir.
7. **Esneklik ve Kullanım Verimliliği:** Kısayollar ve mimikler (gestures) sunulmalıdır.
8. **Estetik ve Sade Tasarım:** Ekranı gereksiz bilgi gürültüsünden arındırmak.
9. **Hataları Teşhis Etme ve Çözme:** Anlaşılır, kodsuz hata mesajları sunmak.
10. **Yardım ve Dokümantasyon:** İhtiyaç anında kolay erişilebilir kılavuzlar sunmak.

4.3. Tasarım Sistemleri ve Rehberler

- **Apple Human Interface Guidelines (HIG):** iOS için akıcı animasyonlar, yuvarlatılmış köşeler ve sadelik odaklı tasarım dili.
- **Google Material Design (Material 3):** Android için katmanlı yapı, dinamik renk sistemleri (Material You) ve belirgin gölgelendirmeler.

4.4. Erişilebilirlik (Accessibility)

Mobil uygulamalar toplumun tüm kesimleri tarafından kullanılabilir olmalıdır:

- **Ekran Okuyucu Desteği:** React Native'de `accessible={true}`, `accessibilityLabel="Giriş Yap Butonu"` tanımları yapılmalıdır.
- **Kontrast ve Metin Boyutu:** Yüksek kontrastlı renk seçimleri ve dinamik metin ölçekleme (Dynamic Type) desteği verilmelidir.

BÖLÜM 5: ÇALIŞMA VE DEĞERLENDİRME SORULARI

Test Soruları

1. **React Native'in yeni mimarisinde asenkron JSON köprüsünün (Bridge) yerini alarak C++ ve JS arasında doğrudan erişim sağlayan katman aşağıdakilerden hangisidir?**

- A) Fabric
- B) JSI (JavaScript Interface)
- C) Web View
- D) TurboModules
- *Cevap: B*

2. **React Native'de `flexDirection` özelliğinin varsayılan değeri nedir?**
    - A) row
    - B) row-reverse
    - C) column
    - D) flex-start
    - *Cevap: C*
3. **Çok sayıda veriden oluşan dinamik bir listede bellek performansını korumak için hangi bileşen tercih edilmelidir?**
    - A) `<ScrollView>`
    - B) `<View>`
    - C) `<FlatList>`
    - D) `<Text>`
    - *Cevap: C*

Doğru / Yanlış Soruları

1. `( D )` React Native uygulamaları bir `WebView` içinde çalışan HTML5 kodlarından ibarettir.
2. `( D )` Expo Prebuild yapısı sayesinde artık Expo projelerine de özel yerel (native) kodlar eklenebilmektedir.
3. `( Y )` React Native'de stil tanımlarken HTML'deki gibi string olarak `"margin-left: 10px"` yazılır. *(Doğrusu: camelCase stil yapısı ile `marginLeft: 10` şeklindedir)*.

Uygulama Ödevi

Bir e-ticaret uygulamasının ürün listeleme ekranını React Native ve Expo kullanarak tasarlayınız.

- **Gereksinimler:**
    1. Verileri ekranda göstermek için `<FlatList>` bileşenini kullanın.
    2. Her ürün kartında ürün görseli (`<Image>`), ürün adı ve fiyatı (`<Text>`) yer alsın.
    3. Tasarımı Flexbox kullanarak düzenleyin (`flexDirection: 'row'`, `alignItems: 'center'`).

4. Performans ve tip güvenliği için `StyleSheet.create()` yapısından yararlanın.